AI WORKFLOW DOCUMENT

How WorkRadar was built with AI

The tools, the prompts that mattered, what AI produced, what changed after review, and the trade-offs behind each decision.

Aditya Singh
Airtribe · AI Product Management
v1.0 · 13 October 2026

01 Overview

AI shows up in WorkRadar in two layers, and the project treats them differently.

AI inside the product. Google Gemini, orchestrated by n8n, writes the manager's daily brief and triages Slack threads and emails, which have no priority or owner fields. It does *not* decide the ranking. A deterministic, explainable score does that, so every item can show why it is on the radar. Gemini runs under a JSON schema, its output is validated, and the app falls back to an on-device summary when no AI is reachable.

AI used to build the product. Claude (Anthropic) was the build partner from brief to delivery. It framed the problem, drafted the PRD, set the visual direction, wrote the React app and the n8n workflows, built a validation harness, and generated the Figma file from the running app. I set the constraints and made the product calls: the Airtribe context, n8n as the workflow engine, self-hosting instead of n8n Cloud, Gemini as the model provider, and the deadline. Every AI output went through a review pass before it shipped.

At a glance

7 AI and automation tools in the workflow

4 n8n workflows: digest, ingestion, actions, morning brief

2 production prompts, both schema-constrained

3 AI engines in the fallback chain: n8n, serverless, on-device

15 hi-fi screens generated from the live app

12 changes made after reviewing AI output (section 06)

02 AI tools used

TOOL	ROLE	WHAT IT DID	WHY THIS TOOL
Claude (Anthropic), agentic coding environment	Build partner	Problem framing, personas, PRD and this document; design direction; React + TypeScript app; scoring model; n8n workflow JSON and Code nodes; validation harness; README; DOM-to-Figma pipeline	Long-context reasoning across product, design and code. It can run, test and screenshot its own work.
Google Gemini · Flash model, Flash-Lite fallback	In-product LLM	Writes the daily brief from a ranked snapshot. Triages unstructured Slack and email threads into priority, waiting-on-manager, blocked, summary, next step and suggested owner.	Native JSON-schema output (`responseSchema`), a free tier for the pilot, low latency
n8n Community Edition, self-hosted in Docker	AI orchestration	Ingests the four sources every 15 min, normalises them, calls Gemini for triage, scores and ranks. Serves the digest and action webhooks, and posts the 08:45 Slack brief.	The assignment required n8n. Self-hosting is free; a Cloudflare tunnel exposes the webhooks.
Vercel Edge Functions	AI gateway	`/api/digest` forwards to n8n when configured, otherwise calls Gemini directly. Keeps the API key server-side.	A free, keyless browser, and a working AI brief even when the laptop running n8n is off
Figma MCP (Plugin API)	Design output	Rebuilt captured app states as editable Figma layers. A bundled plugin covers the remaining screens.	Design file and shipped product never drift
Playwright (headless Chromium)	Capture and QA	Renders 15 app states for the DOM extractor and screenshots, visual review at desktop and mobile widths, HTML-to-PDF for these documents	Repeatable visual checks without manual clicking
GitHub Actions	Verification	Typecheck, production build and the n8n simulator on every push	Catches AI-introduced regressions automatically

03 How the work flowed

1 · FRAME
Problem and users
Turned the brief into four manager questions, two personas and six pain points.
AI: drafts · **Me:** Airtribe context, scope

2 · DECIDE
Product calls
Explainable ranking, AI for language only, three-engine fallback, mock data for the demo.
AI: options and trade-offs · **Me:** chose

3 · DESIGN
Visual direction
"Air-traffic control": radar scope hero, flight-strip rows, Barlow type, light and dark tokens.
AI: direction and CSS · **Me:** approved

4 · BUILD
App and workflows
React app, scoring, search parser, 4 n8n workflows, Vercel routes, 47 mock items relative to "now".
AI: code · **Me:** keys, Docker, repo

5 · VERIFY
Review loops
Screenshot review against the PRD, an n8n simulator, CI, API route tests. 12 changes.
AI: self-review · **Me:** acceptance

6 · SHIP
Deliverables
Figma from the live app, PRD, this document, README, deploy and n8n guides.
AI: drafts · **Me:** final edit, demo

04 Where AI runs inside the product

SOURCES	N8N · SOURCE INGESTION (EVERY 15 MIN)	BRIEF GENERATION	MANAGER
Slack Threads, DMs, mentions. No priority field.	**Normalise** 4 payloads → one WorkItem shape (owner, due, links, blockers)	**n8n · Daily digest · AI** The ranked snapshot (top 24 open, blocked people, last 24h) goes to Gemini with a JSON schema, then a validation step	**WorkRadar app** Today, Priority feed, Item detail, Search, Team. Every score shows "why".
Jira Tickets with P-levels, sprints, links	**Triage unstructured threads · AI** Gemini reads Slack and email and returns priority, waitingOnManager, blocked, escalation, summary, nextStep and suggestedOwner	**Vercel /api/digest** Same prompt and schema when n8n is unreachable. Same-origin only, 80 KB cap, Flash-Lite retry on 429/404/503.	**Actions → n8n** Assign, Done and Snooze post to the action webhook (demo mode logs; live mode writes back)
GitHub PRs, reviews, checks	**Score and rank** 6 weighted signals → 0–100, P0 pinned ≥ 97, rings: now ≥ 75, today ≥ 50	**On-device summary** A rule-based brief from the same snapshot, so the brief is never empty. The engine is always labelled.	**08:45 Slack brief · AI** An n8n cron job calls the items and digest webhooks and posts five lines to Slack
Gmail Escalations, stakeholder asks			

Green boxes use an LLM. Grey boxes are deterministic code. The split is deliberate: AI handles language and ambiguity, code handles ranking and anything that must be auditable.

05 Significant prompts

A · Daily brief

Where: n8n "Daily digest" and `api/_prompt.ts`, as one shared text

Model: Gemini Flash, temperature 0.3, `responseMimeType: application/json` + `responseSchema`

Design choices: facts-only rule, exact IDs so output can be validated, sentence-length caps so it fits the UI, and a fixed shape for each of the five sections.

```
You are WorkRadar, a chief-of-staff assistant for an engineering manager.
You receive a JSON snapshot of their team's work from Slack, Jira, GitHub and email.
Items are already ranked by a transparent priority score (0-100) and grouped into rings:
"now" (needs attention now), "today" and "later".
Write the manager's brief as JSON that matches the response schema.

Rules
- Use only facts in the snapshot. Never invent people, numbers, deadlines or items.
- Put the exact item "id" from the snapshot in every itemId field. Use team member ids for
personId.
- Plain, specific language. No hype, no emojis, no exclamation marks.
- Each "why", "action" and "suggestion" is one sentence under 25 words.

Fields
- headline: 1-2 sentences. How many items need attention, how many people are blocked
  (and how many on the manager), and the one thing to start with and why.
- topWork: exactly 3 items the manager should personally act on today ... "why" is the concrete
next step.
- happened: up to 6 notable events from happenedToday as short past-tense sentences.
- attention: every item in ring "now" (max 6), with why it matters now and the action to take.
- blocked: one entry per blocked person with personId, itemId, blockedOn, since and suggestion.
```

B · Thread triage

Where: n8n "Source ingestion", Prepare AI triage → Gemini

Model: Gemini Flash, temperature 0.2, schema with a `P0–P3 | NONE` enum and booleans. The results are merged into each item's signals before scoring.

Design choices: Airtribe-specific P-level definitions anchored on learners and revenue; the team roster plus skills, for owner suggestions; FYIs filtered out before the call to save tokens.

```
You triage workplace messages for Neha Kapoor, Engineering Manager of the Learning Platform team
at Airtribe (an online learning company running live cohort programs).
For every thread, return one object in "results" with:
- priority: P0 (live incident hurting learners now), P1 (urgent, learners or revenue affected),
  P2 (normal), P3 (low), or NONE for FYI
- waitingOnManager: true only if someone is explicitly waiting for Neha to decide, review or
reply
- blocked: true if a teammate says they cannot proceed
- escalation: true for incidents or escalations from leadership or customers
- summary: one plain sentence under 30 words with the key facts (numbers, deadlines)
- nextStep: one sentence under 25 words telling the manager what to do
- suggestedOwner: the team member id best placed to own it, or "" if it is the manager's own
decision
Use only the facts in the text. Team: [{"id":"rahul","role":"Senior Frontend Engineer","skills":
[...]}, ...]
```

Prompts are quoted from the repository (`api/_prompt.ts`, `n8n/src/ingest-classify-request.js`); "…" marks shortening.

05 Significant prompts, continued: directing the build

C · The project brief given to Claude (condensed)

It set the deliverables and the quality bar. Its last line made Claude plan before building, which surfaced the questions that mattered: the author, the target company, the deadline, n8n hosting and Figma access.

```
Build WorkRadar, an AI-powered web app that helps
managers see top work for the day, what happened today,
what needs immediate attention and who is blocked,
without checking Slack, Jira, GitHub or email.

Deliverables: PRD (PDF, max 2 pages) · hi-fi UI designs
(Dashboard, Priority Feed, Item Details, Search,
Empty/Error state) · working MVP (dashboard, AI
summary, priority feed, search, detail page, Assign /
Mark Done / Snooze; mock data is fine) · AI workflow
doc (tools, significant prompts, what AI generated,
what was modified, trade-offs) · source code · demo
video.

Use n8n for workflows. I want a full-fledged,
functioning, visually aesthetic project, so use your
best judgement for the design tool. Before starting,
prepare a list of tasks or things you need from me, and
deliver the entire project with maximum value.
```

D · Framing and constraints (my reply, lightly edited)

One reply reshaped the product. The Airtribe context gave the mock data its stakes: live classes, the GenAI Launchpad and mentor sessions. "No n8n Cloud" led to self-hosted n8n plus a serverless fallback.

```
1. Aditya Singh. Create this project for an Airtribe
manager itself, for best impact, as this assignment is
part of my AI product management course. Their website:
airtribe.live
2. The assignment is due on 16 Oct 2026.
3. Give me the steps for the API setup for n8n with
Gemini.
4. I don't have an n8n Cloud subscription. Choose the
next best alternative.
5. I have connected GitHub with Claude, with an empty
repo called workradar.
6. Once the project is complete, ask me again (about
the demo video).
7. Yes, you can create a new file in my Figma drafts.
```

How review worked in the build loop: each screen was screenshotted and reviewed from a sceptical manager's point of view at 9 am, asking what is wrong, confusing or untrustworthy. Every number in the brief was checked against the snapshot.

06 What AI generated, and what changed after review

AREA	GENERATED BY AI	CHANGED AFTER REVIEW
Ranking model	Six-signal weighted score, rings, "why" details per factor	→ The P0 iPhone incident ranked below a PR, so P0 is now pinned to ≥ 97. → Slack and email items read "No priority set". They now read "Inferred P1".
Brief copy	Prompt, schema, on-device brief, engine labels	→ Absolute clock times contradicted relative times ("in 5h"), so they were removed from AI copy. → "1 days" became "about a day", and lowercase sentence starts were capitalised. → The engine label now says "Gemini via n8n" or "n8n (rule-based)".
Validation	JSON schema output	→ Added `sanitizeBrief` : drops unknown item IDs, resolves names to people, caps list lengths, rejects briefs without a headline.
Priority feed	Flight-strip rows, views, filters, keyboard triage	→ The default view was too sparse, so it is now "All open" with ring headers. → Item refs were truncated, so columns were widened and DM refs shortened. Keyboard hints moved to their own line.
Empty and error states	First run, no results, outage banner, AI error with retry	→ On first run, source icons stretched and the sidebar claimed sources had synced "2m ago". Fixed to "Not connected".
n8n workflows	4 workflows as JSON, 9 Code-node sources, mock source payloads	→ The n8n docs weren't reachable, so node versions and parameters were checked against n8n's source (nodes-base 2.44). Added a simulator to CI that replays the whole pipeline.
Security	Server-side Gemini route	→ Added a same-origin check, an 80 KB payload cap, a 40-item cap and a model fallback. The key lives only in n8n credentials and Vercel env, never in code or chat.
Figma pipeline	DOM extractor and Figma renderer	→ Fixed text that truncated when it fit, missing underlines and percentage corner radii. Starter-plan limits led to a bundled plugin.

Written by hand, not generated: the choice of problem and users, the Airtribe framing, all product scope decisions in the PRD, the n8n hosting decision, API keys and accounts, and final acceptance of each deliverable.

Prompting patterns that worked

- **Ask for a plan and questions first.** "Prepare a list of what you need from me" turned a vague brief into five concrete decisions before any code was written.
- **Give the model the business definitions.** P0–P3 anchored on learners and revenue made triage consistent with how Airtribe actually prioritises.
- **Constrain output to the UI.** Schema fields map one-to-one to UI slots, with word limits sized to the layout.
- **Make hallucinations detectable.** Requiring exact item IDs means any invented item can be dropped automatically.
- **One prompt, many runtimes.** The brief prompt lives in one file that n8n and Vercel both use, so the engines can't drift.
- **Review against screenshots, not code.** Most quality issues (ranking surprises, copy, truncation) were only visible on the rendered screen.

07 Key trade-offs

DECISION	CHOSEN	INSTEAD OF	WHY, AND WHAT IT COSTS
Who ranks	Deterministic weighted score; AI for language and triage	LLM ranks everything	Explainable, stable between refreshes, testable and free. It can't pick up subtle context, so Gemini triage feeds it signals and the weights are adjustable.
Output format	JSON schema + validation	Free-text summary	Renders into specific UI slots, links to real items and can be checked for invented IDs. Costs some expressiveness, mitigated by tight per-field instructions.
Orchestration	Self-hosted n8n CE (Docker + Cloudflare tunnel)	n8n Cloud; plain backend code	Meets the n8n requirement at zero cost, and the workflows are visual for a PM audience. The tunnel URL changes and the laptop must be on, hence the next row.
Availability	n8n → Vercel Gemini → on-device chain	Single AI path	The brief is never empty and the engine is always labelled. Costs one prompt kept in two places, solved with a single shared prompt file.
Model size	Gemini Flash, with Flash-Lite on 429/404/503	Pro-class model	The brief summarises an already-ranked snapshot, so a fast model suffices. Lower latency, within the free tier. Nuance on long threads is weaker.
Data sent to AI	Top 24 open items + blocked people + last 24h	Every raw thread	Fewer tokens, less private content leaving the system, more focused output. Low-ranked items can't appear in the brief, which is acceptable by design.
Data	Mock workspace with timestamps relative to "now"	Live OAuth connectors	Allowed by the brief. The demo always reads as "today" and is safe to share. Real-world noise and edge cases are untested until the pilot.
Design file	Generated from the running app (DOM → Figma)	Mockups drawn first	Design and code never drift, and 15 states came cheaply. Layers are absolutely positioned rather than auto-layout components.
UI stack	Hand-written CSS tokens, no UI library	Component library	A distinctive look, a small bundle (85 KB gzip JS) and full control of light and dark. More code to maintain; AI made it cheap to write.

Guardrails and how the AI is checked

- **Facts-only prompting** with exact IDs, then **validation** that drops anything the snapshot can't support.
- **Every score is explained** by factor in the UI, so the manager can disagree and re-weight.
- **An engine label on every brief** (Gemini, Gemini via n8n, n8n rule-based, on-device), so users know what they're reading.
- **The pipeline simulator** (`n8n/validate.mjs`) replays ingestion → triage → score in CI. The expected top three are the iPhone incident (97), PR #1284 (93) and LP-512 (86).
- **Graceful failure:** the outage banner hides a down source and explains the effect on ranking. AI errors offer a retry or the on-device summary.
- **Secrets** stay only in n8n credentials and Vercel env. The browser never sees the key.

What I'd do next with AI

- **An eval set:** 50 golden snapshots with expected top-3 lists, plus an LLM-as-judge faithfulness check on every brief, tracked weekly.
- **Learned weights:** tune the six weights per manager from opens, snoozes and reorders, keeping the explanation.
- **Reply drafts:** Gemini drafts responses for "waiting on you" asks, sent only after the manager approves.
- **Write-back** through n8n: Jira assign, GitHub review requests and Slack replies, with an audit log and undo.
- **Cost and latency dashboards** from n8n execution data, with budget alerts per team.
- **A privacy layer:** redact personal data before triage, and keep message bodies for at most 7 days.

Lesson The most useful AI work wasn't generating code. It was generating options and then critiquing the result from the user's point of view. The biggest quality jumps came from review passes against real screenshots, not from longer build prompts.

Links Source and workflows: github.com/adityasingh535402-del/WorkRadar · Designs: figma.com/design/Q8RHXzlj6yOqBR4HDq5hn5 · PRD: `docs/WorkRadar-PRD.pdf`